

Quiz #2

CS 225 Data Structures

Spring 2026

7:00-8:00pm, Thursday, March 12

Name	Hanbin Zheng
ZJU ID	
Lab TA	

- This is a **closed book** and **closed notes** exam. No electronic aids are allowed, either.
- You should have 4 problems total on 11 pages. You must write on the exam.
- Unless otherwise stated in a problem, assume the best possible design of a particular implementation is being used.
- Unless the problem specifically says otherwise, (1) assume the code compiles, and thus any compiler error is an exam typo (though hopefully there are not any typos), and (2) assume you are **NOT** allowed to write any helper methods to help solve the problem, nor are you allowed to use additional arrays, lists, or other collection data structures unless we have said you can.
- Please put your name at the top of each page.
- Please fill in your **Lab Section** or your **Lab TA's name** to help us organize and grade your papers.
- Don't panic and good luck!

Problem 1	<u>8.5</u>	/12
Problem 2	<u>10</u>	/10
Problem 3	<u>9</u>	/9
Problem 4	<u>9</u>	/9
Total	<u>36.5</u>	/40



Problem 1 (12 points): LABs & MPs

1. Warming up!!!

Suppose you have the following code:

Person.h

```
...
class Person {
private:
    string name_;
public:
    Person(const string& name);
    string getName() const;
};
class Student : public Person {
private:
    int id_;
    int* score_; // dynamically allocated integer
public:
    Student(const string& name, int id, int score);

// TODO1: Write the declarations required for Part 2 here.

    int getId() const;
    int getScore() const;
    void setScore(int score);
};
...
```

没有实现 destructor 的
virtual. 多费题反

Person.cpp

```
...
Person::Person(const string& name)
    : name_(name)
{
}
string Person::getName() const {
    return name_;
}
Student::Student(const string& name, int id, int score)
    : id(id)
{
    score_ = new int(score);
}
int Student::getId() const {
    return id_;
}
int Student::getScore() const {
    return *score_;
}
void Student::setScore(int score) {
```

Student new/delete ★



```

    *score_ = score;
}
// TODO2: Write the definitions required for Part 2 here.

```

There is one bug in the code above. Underline the incorrect line(s) and write the corrected version below. You do not need to use all the blank lines provided.

```

Student::Student(const string& name, int id, int score) :
    Person(name), id(id) { score_ = new int;
                          *score_ = score; }

```

2. (Assume the bug is fixed) You may notice that `score_` is dynamically allocated. Therefore, the `Student` class must manage it correctly according to the Rule of Three. Complete the missing code in both `Person.h` and `Person.cpp` by writing:

- (1) destructor
- (2) copy constructor
- (3) copy assignment operator

You may write "IDK" for 1 point, but no additional credit will be given for this part in that case.

(Write IDK here -> _____)

Hint:

```

int* value;
value = new int(4); // dynamically allocates an int with value 4

```

`Person.h`

// TODO1 : Write the declarations required for Part 2 here.

```

virtual ~Person() = default; // This line should be add to Person class

```

```

~ Student();
Student(const Student & other);
Student& operator = (const Student & other);

```

// You should declare all three member functions.

`Person.cpp`

// TODO2: Write the definitions required for Part 2 here.

```

Student::~~Student() { delete score_; score_ = nullptr; }
Student::Student(const Student & other) : Person(other.getName(),
    id(other.id)) { score_ = new int;
                  *score_ = other.score_; }
Student& Student::operator = (const Student & other) {
    name_ = other.getName();
    id_ = other.id;
    *score_ = other.getScore();
}

```

① `&other == this`? 不能 `*this == other`

② 调用 base class in operator= , 因为 `name_` 是 private, 只能这么干



Name:

4

//Extra blank lines are provided in case you need more space.



Problem 2 (10 points): C++ Special Class Members

1. Consider the following code:

```

class Sphere {
public:
    Sphere();
    Sphere(const Sphere &s);
    Sphere& operator=(const Sphere &s);
};
Sphere s1, s2;
s2 = s1; assignment
Sphere s3 = s1; copy constructor

```

Which statement is correct?

- a) The copy constructor is called twice
- b) The assignment operator is called twice
- c) The assignment operator is called once and the copy constructor is called once
- d) Only the default constructor is called

2. Consider the following code:

```

class Sphere {
public:
    Sphere() {}
    Sphere(const Sphere &s) {}
};
Sphere s1;
Sphere s2 = s1;
Sphere *s3 = &s1;
Sphere &s4 = s2;

```

How many times is the copy constructor called?

- a) 0
- b) 1
- c) 2
- d) 3

3. Consider the following class:

```

class Sphere {
public:
    Sphere() {
        props = new std::string[5];
    }

    ~Sphere() {
        delete [] props;
    }
}

```



```
private:
    std::string *props;
};
```

b. If the compiler-generated copy constructor is used, what problem may occur?

- ~~a) Memory leak~~
- b) Double deletion of the same memory ✓
- ~~c) The destructor will never be called~~
- ~~d) The program will not compile~~

b. 4. According to the Rule of Three, if a class defines one of the following functions, it usually needs to define all three:

- a) Constructor, destructor, and operator+
- b) Copy constructor, assignment operator, and destructor
- c) Constructor, operator+, and operator<<
- d) Destructor, operator[], and operator+

5. Consider the following code:

```
class Sphere {
public:
    Sphere() {}
    ~Sphere() {}
};

int main() {
    Sphere s1;
    Sphere s2;
    Sphere *s3 = new Sphere;

    delete s3; ①

    return 0;
}
```

c. How many times will the destructor be called?

- a) 1
- b) 2
- c) 3
- d) 4



Problem 3 (9 points): object lifecycle & parameter**1. Considering the following code:**

```
#include <iostream>
using namespace std;

class Number {
public:
    Number(int v = 0) {
        data = new int(v);
        cout << "C" << endl;
    }

    Number(const Number& other) {
        data = new int(*other.data);
        cout << "CC" << endl;
    }

    ~Number() {
        cout << "D" << endl;
        delete data;
    }

    Number& operator=(const Number& other) {
        cout << "A" << endl;
        if (this != &other) {
            delete data;
            data = new int(*other.data);
        }
        return *this;
    }

    Number operator+(const Number& other) const {
        Number temp(*data + *other.data);
        return temp;
    }

    Number& addAssign(const Number& other) {
        *data += *other.data;
        return *this;
    }

    int get() const {
        return *data;
    }

private:
    int* data;
};
```



```

Number make(Number x) {
    Number y(10);
    x = x + y;
    return x;
}

```

```

Number& bad(Number x) {
    Number y(100);
    x.addAssign(y);
    return x;
}

```

x : tempt, on the stack.

$$\frac{b+c}{b} = 2+11=13$$

```

int main() {
    Number a(1); ✓
    Number b(2); ✓
    Number c = make(a); ✓
    a = b + c; ✓
    Number& r = bad(b);
    cout << a.get() << " " << c.get() << endl;
    cout << r.get() << endl;
    return 0;
}

```

$$C = (\cancel{\text{make}(1)} + 10) + 10 = 11$$

$a \leftarrow 13$

(1) Why the code above does not compile? identify the problematic function and explain why. (At most 2 sentence is sufficient)

In function `Number& bad (Number x)`, it returns the object x , which is on the stack and will be destroyed after the return.

(2) Ignoring the problematic function for a moment, and assuming `main()` only includes the following lines:

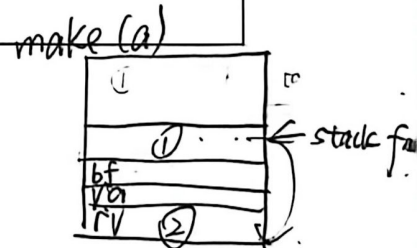
```

Number a(1);
Number b(2);
Number c = make(a);
a = b + c;
cout << a.get() << " " << c.get() << endl;

```

Fill in the counts:

Constructor called: 5
 Copy constructor called: 5
 Assignment operator called: 2



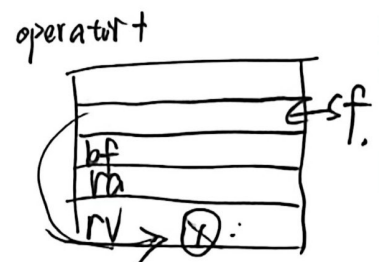
(3) Under the same assumption as Part 2, what is the final output of:

```

cout << a.get() << " " << c.get() << endl;

```

13 11



Name:

9

2. Considering the following code:

```
#include <iostream>
using namespace std;

void mystery(int a, int& b, int* c) {
    a = b + *c;
    b = a - b;
    *c = a - *c;
    b = b + *c;
}

int main() {
    int x = 2;
    int y = 4;
    mystery(x, y, &x);
    cout << x << " " << y << endl;
    return 0;
}
```

$$b \leftarrow *c + b$$

$$*c \leftarrow b$$

$$a \leftarrow b + *c$$

$$b \leftarrow (b + *c) - b = *c$$

$$*c \leftarrow (b + *c) - (*c) = b$$

$$b \leftarrow (*c) + (b)$$

$$x \leftarrow \boxed{2}$$

$$y \leftarrow \boxed{4}$$

What is the output?

x = 4

y = 6

$$x \leftarrow 2$$

$$y \leftarrow 4 + 2 = 6$$

$$y \leftarrow 4 + 2$$

$$x \leftarrow 4$$

$$a \leftarrow b + *c$$

$$b \leftarrow (b + *c) - b = *c$$

$$*c \leftarrow (b + *c) - *c = b$$

$$b \leftarrow (*c) + b$$



Problem 4 (9 points): Inheritance & Polymorphism, Virtual Function Table Pointer**1. Consider the following code:**

```

#include <iostream>
#include <string>
using namespace std;

class Device {
public:
    Device(string n) : name(n) {}
    virtual void info() const {
        cout << name << " device" << endl;
    }
    virtual int power() const = 0;
private:
    string name;
};

class Laptop : public Device {
public:
    Laptop(string n, int b) : Device(n), battery(b) {}
    void info() const {
        cout << "Laptop battery = " << battery << endl;
    }
    int power() const {
        return battery * 2;
    }
private:
    int battery;
};

class Phone : public Device {
public:
    Phone(string n, int s) : Device(n), signal(s) {}
    int power() const {
        return signal + 1;
    }
private:
    int signal;
};

int main() {
    Laptop l("ThinkPad", 5);
    Phone p("iPhone", 3);

    Device* ptr1 = &l;
    Device* ptr2 = &p;

    ptr1->info();
    ptr2->info();
}

```



```

cout << ptr1->power() << endl;
cout << ptr2->power() << endl;

return 0;
}

```

(1) What will be printed to the screen? Write the output in order.

Laptop battery = 5
 iphone device
 10
 4

(2) Is it legal for us to create an object directly using Device d("abc")?

No. Since Device has a pure virtual function, and thus it's an abstract class, which cannot have any object.

- b (3) Which of the following statements is correct?
- a) ptr1->info(); always calls Device::info() because ptr1 is a Device*
 - b) ptr2->power(); calls Phone::power() at runtime
 - c) Laptop cannot inherit from Device because Device has a constructor
 - d) Phone must override info() or the program will not compile

2. Given the following class template:

```

template <class T>
class Pair {
public:
    Pair(T a, T b);
private:
    T x;
    T y;
};

```

A Pair object stores two values of the same type. When a Pair object is created, its constructor should assign the first argument to x and the second argument to y.

Complete the definition of the constructor for Pair by filling the blanks:

```

template <class T>
Pair<T>::Pair(T a, T b) {
    x = a;
    y = b;
}

```

